

Relational Calculus

Definition

- A **relational calculus** expression
 - creates a new relation,
 - which is specified in terms of variables that
 - range over rows of the stored database relations (in **tuple calculus**)
 - or over columns of the stored relations (in **domain calculus**).

Relational Calculus

- In a calculus expression, there is *no order of operations* to specify how to retrieve the query result
 - a calculus expression specifies only what information the result should contain.
 - This is the main distinguishing feature between relational algebra and relational calculus.
- Relational calculus is considered to be a **nonprocedural** language.
- This differs from relational algebra, where we must write a *sequence of operations* to specify a retrieval request;
- hence relational algebra can be considered as a **procedural** way of stating a query.

Tuple Relational Calculus

- The tuple relational calculus is based on specifying a number of **tuple variables**.
- Each tuple variable usually *ranges over* a particular database relation, meaning that the variable may take as its value any individual tuple from that relation.
- A simple tuple relational calculus query is of the form
 $\{t \mid R(t) \text{ COND}(t)\}$
where t is a tuple variable and $\text{COND}(t)$ is a conditional expression involving t . The result of such a query is the set of all tuples t that satisfy $\text{COND}(t)$.

Tuple Relational Calculus

- **Example:** To find the first and last names of all employees whose salary is above \$50,000, we can write the following tuple calculus expression:

Query-1:

{t.FNAME, t.LNAME | EMPLOYEE(t) AND t.SALARY>50000}

The condition EMPLOYEE(t) specifies that the **range relation** of tuple variable t is EMPLOYEE. The first and last name (PROJECTION $\pi_{\text{FNAME}, \text{LNAME}}$) of each EMPLOYEE tuple t that satisfies the condition t.SALARY>50000 (SELECTION $\sigma_{\text{SALARY} > 50000}$) will be retrieved.

The Existential and Universal Quantifiers

- Two special symbols called **quantifiers** can appear in formulas; these are the **universal quantifier** $\forall$) and the **existential quantifier** $\exists$).
- Informally, a tuple variable t is **bound** if it is quantified, meaning that it appears in an $(\forall t)$ or $(\exists t)$ clause; otherwise, it is **free**.
- If F is a formula, then so is $(\exists t)(F)$, where t is a tuple variable. The formula $(\exists t)(F)$ is true if the formula F evaluates to true for **some (at least one)** tuple assigned to free occurrences of t in F ; otherwise $(\exists t)(F)$ is **false**.

The Existential and Universal Quantifiers

- If F is a formula, then so is $(\forall t)(F)$, where t is a tuple variable. The formula $(\forall t)(F)$ is true if the formula F evaluates to true for *every tuple* (in the universe) assigned to free occurrences of t in F ; otherwise $(\forall t)(F)$ is **false**.
- It is called the universal or “for all” quantifier because every tuple in “the universe of” tuples must make F true to make the quantified formula true.

$\exists$ Existential Quantifier (“there exists”)

General Form

$$\{ t \mid \exists s \text{ (RELATION}(s) \wedge \text{Condition}(s, t)) \}$$

Example

Assume we have two relations:

- **Employee**(emp_id, name, dept_id)
- **Department**(dept_id, dept_name)

We want all employees who work in the ‘Sales’ department:

$$\{ e.name \mid \text{Employee}(e) \wedge \exists (d) ((\text{Department}(d) \wedge d.dept_name = 'Sales' \wedge e.dept_id = d.dept_id)) \}$$

$\forall$ Universal Quantifier (“for all”)

General Form

$\{ t \mid \forall s \text{ (Condition}(s, t)) \}$

Example

Suppose we want all employees who **work on every project** in the company.

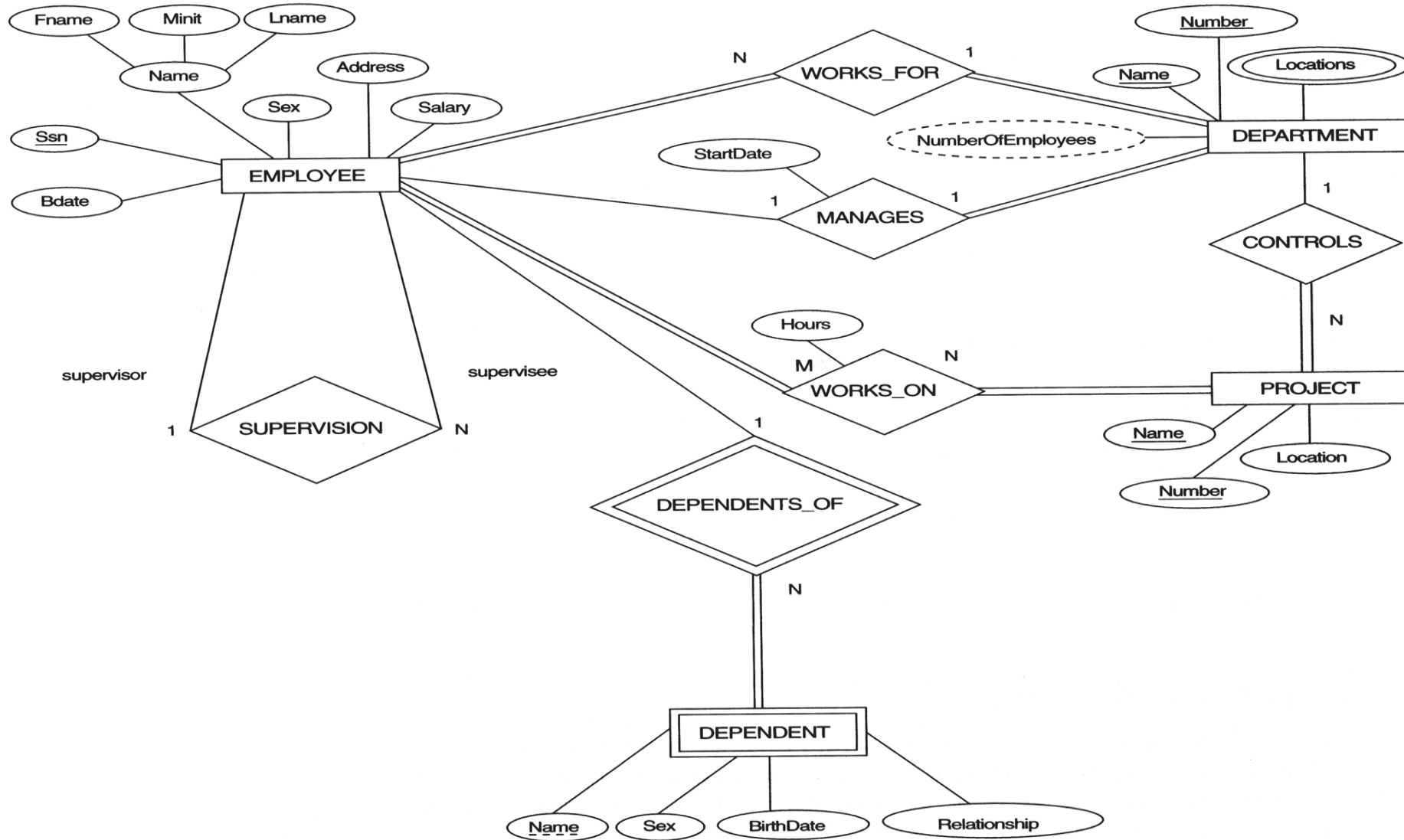
Relations:

- Employee(emp_id, name)
- Project(proj_id)
- WorksOn(emp_id, proj_id)

Query:

$\{ e.name \mid \text{Employee}(e) \wedge \forall (p) (\text{Project}(p) \wedge \exists (w) (\text{WorksOn}(w) \wedge w.emp_id = e.emp_id)) \}$

Exercise



Sample

Retrieve the first name and address of all employees who manages the 'Financial' department.

Query:

**{e.FNAME, e.ADDRESS | EMPLOYEE(e) and ($\exists$ d)
(DEPARTMENT(d) and d.DNAME='Financial' and d.DNUMBER=e.DNO) }**

- The *only free tuple variables* in a relational calculus expression should be those that appear to the left of the bar (|). In above query, t is the only free variable; it is then *bound successively* to each tuple. If a tuple *satisfies the conditions* specified in the query, the attributes FNAME and ADDRESS are retrieved for each such tuple.
- The conditions EMPLOYEE (e) and DEPARTMENT(d) specify the range relations for e and d. The condition d.DNAME = 'Financial' is a selection condition and corresponds to a SELECT operation in the relational algebra, whereas the condition d.DNUMBER = e.DNO is a JOIN condition.

Exercise

Task:

2. Retrieve the name, salary and address of all employees who work for the 'Research' department.

Exercise

Task:

2. Retrieve the name, salary and address of all employees who work for the 'Research' department.

Query-2:

**{e.FNAME, e.LNAME, e.Salary, e.ADDRESS | EMPLOYEE(e) and ($\exists d$)
(DEPARTMENT(d) and d.DNAME='Research' and d.DNUMBER=e.DNO) }**

Exercise

Task:

Find the names of employees who Not works on all projects controlled by department number 5.

Exercise

Query 3: Find the names of employees who Not works on all projects controlled by department number 5.

**Q3: { e.FNAME, e.LNAME | EMPLOYEE(e) $\wedge$
not ($\forall p$ ((PROJECT(p) $\wedge$ p.DNUM = 5)
 $\wedge$ $\exists(w)$ (WORKS_ON(w) $\wedge$ w.ESSN = e.SSN $\wedge$ w.PNO =
 p.PNUMBER)
)
)
}**

Transforming

- Well-known transformations from mathematical logic
 - $(\forall x)(P(x)) \equiv (\text{not } \exists x)(\text{not } (P(x)))$
 - $(\exists x)(P(x)) \equiv \text{not } (\forall x)(\text{not}(P(x)))$
 - $(\forall x)(P(x) \text{ and } Q(x)) \equiv (\text{not } \exists x)(\text{not}(P(x)) \text{ or } \text{not } (Q(x)))$
 - $(\forall x)(P(x) \text{ or } Q(x)) \equiv (\text{not } \exists x)(\text{not}(P(x)) \text{ and } \text{not } (Q(x)))$
 - $(\exists x)(P(x) \text{ or } Q(x)) \equiv \text{not}(\forall x)(\text{not}(P(x)) \text{ and } \text{not } (Q(x)))$
 - $(\exists x)(P(x) \text{ and } Q(x)) \equiv \text{not}(\forall x)(\text{not}(P(x)) \text{ or } \text{not } (Q(x)))$

Transforming (cont)

- The following is also true, where $\Rightarrow$ stands for implies:
 - $(\forall x)(P(x)) \Rightarrow (\exists x)(P(x))$
 - $(\text{not } \exists x)(P(x)) \Rightarrow \text{not } (\forall x)(P(x))$
- The following is *not true*:
 - $\text{not}(\forall x)(P(x)) \Rightarrow (\text{not } \exists x)(P(x))$

Example Transforming

Additional examples:

Query 6: Find the names of employees without any dependents

**Q6: {e.FNAME, e.LNAME | EMPLOYEE(e) and
(not($\exists d$)(DEPENDENT(d) and e.SSN = d.ESSN)))}**

Transforming Q6 into Q6' to use universal quantifier

**Q6': {e.FNAME, e.LNAME | EMPLOYEE(e) and
(($\forall d$)(not(DEPENDENT(d)) or not(e.SSN = d.ESSN)))}**

Other Example Transforming

Query 7:

List the names of managers who have at least one dependent

**Q7: {e.FNAME, e.LNAME | EMPLOYEE(e) and
($\exists d$)($\exists p$)(DEPARTMENT(d) and DEPENDENT(p) and
e.SSN = d.MGRSSN and p.ESSN = e.SSN)}**

Transforming Q7 into Q7' to use universal quantifier ?

Language based on Tuple Rel Calculus

- The language **SQL** is based on tuple calculus. It uses the basic

SELECT <list of attributes>

FROM <list of relations>

WHERE <conditions>

block structure to express the queries in tuple calculus where the SELECT clause mentions the attributes being projected, the FROM clause mentions the relations needed in the query, and the WHERE clause mentions the selection as well as the join conditions.

- Another language which is based on tuple calculus is **QUEL** which actually uses the range variables as in tuple calculus.

Its syntax includes:

RANGE OF <variable name> IS <relation name>

Then it uses

RETRIEVE <list of attributes from range variables>

WHERE <conditions>

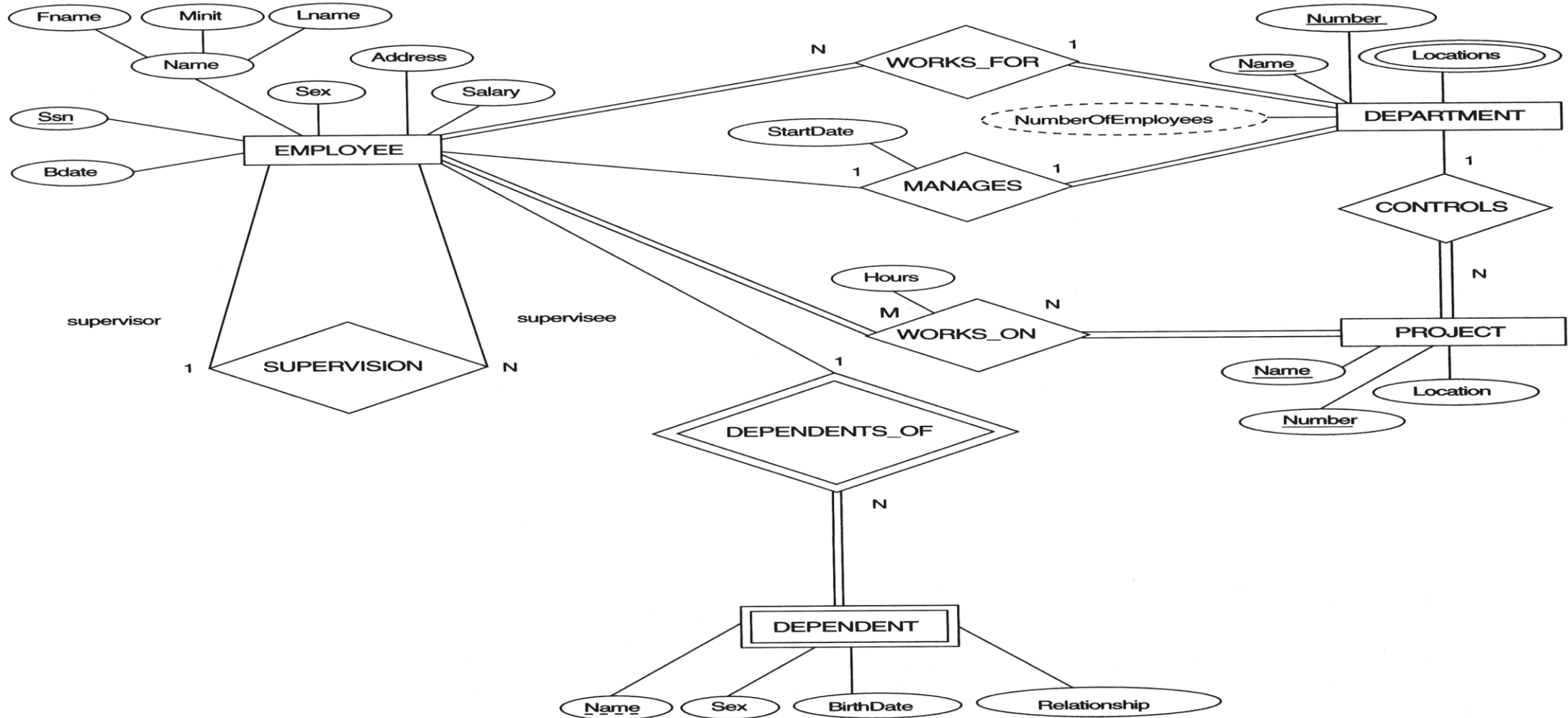
This language was proposed in the relational DBMS INGRES.

Domain Rel Calculus

- Variation of relational calculus called the domain relational calculus, or simply, **domain calculus** is equivalent to tuple calculus and to relational algebra.
- The language called QBE (Query-By-Example) that is related to domain calculus was developed almost concurrently to SQL at IBM Research, Yorktown Heights, New York.
 - Domain calculus was thought of as a way to explain what QBE does.

- Domain calculus differs from tuple calculus in the *type of variables* used in formulas:
 - rather than having variables range over tuples, the variables range over single values from domains of attributes.
 - To form a relation of degree n for a query result, we must have n of these **domain variables**—one for each attribute.
- An expression of the domain calculus is of the form
 $\{x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m})\}$
where $x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m}$ are domain variables that range over domains (of attributes) and COND is a **condition** or **formula** of the domain relational calculus.

Task



Task

Schema (assumed)

- a. **EMPLOYEE**(FNAME, MINIT, LNAME, SSN, SALARY, DNO, ...)
- b. **DEPARTMENT**(DNAME, DNUMBER, MGRSSN, ...)
- c. **PROJECT**(PNAME, PNUMBER, PLOCATION, DNUM)
- d. **WORKS_ON**(ESSN, PNO, HOURS)
- e. **DEPENDENT**(ESSN, DEPENDENT_NAME, ...)

Question

1. Employees who work on all projects located in 'Houston'
2. Find the names of employees who are **supervisors** of at least one other employee.
3. Find employees who are **not assigned** to any project that is controlled by the same department they work in.
4. Find the names of **managers** who have dependents and whose total weekly work hours exceed 40



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,
niscaya ilmu yang sedikit pun akan bermanfaat baginya